

Transformer from Scratch

Technical Report for FIT BUT NLP course (ZPJ a)

David Kedra, xkedra00

January 20, 2026

1 Introduction

The Transformer architecture introduced in *Attention Is All You Need* [1] has become the foundation of modern NLP, and the goal of this project was to understand it in detail, reimplement it from scratch, and demonstrate its functionality. The implementation was restricted to the use of basic neural network primitives such as linear transformations, normalization layers, and simple activation functions, without relying on prebuilt high-level modules.

The project focuses on the complete encoder–decoder Transformer architecture and its application to sequence-to-sequence tasks. The model was trained and evaluated on a real-world machine translation task using publicly available datasets. In addition, the model was also tested on a question answering dataset to verify its performance on another task.

Besides reproducing the original Transformer architecture, the project explores different approaches to positional encoding and compares the standard sinusoidal positional encoding with the Rotary Position Embedding (RoPE) [2] implemented according to the original paper.

2 Sequence-to-Sequence Task Explanation

The Transformer is a neural network architecture that can be used as a generative model for sequence-to-sequence tasks. Given a source sequence of tokens

$$\mathbf{x} = (x_1, x_2, \dots, x_n), \quad (2.1)$$

the model aims to generate a target sequence

$$\mathbf{y} = (y_1, y_2, \dots, y_m), \quad (2.2)$$

by modeling the conditional probability of the target sequence

$$p(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^m p(y_t \mid y_{<t}, \mathbf{x}), \quad (2.3)$$

where $y_{<t}$ denotes all previously generated target tokens, and in practice the decoder input is prepended with a start-of-sequence token “<SOS>” to provide context for predicting y_1 .

In machine translation, the source sequence corresponds to a sentence in the source language (Czech), and the target sequence corresponds to its translation in the target language (English).

Unlike recurrent neural networks, the Transformer does not process tokens sequentially. Instead, it uses self-attention layers to capture dependencies among all tokens simultaneously by allowing every token in a sequence to attend directly to all other tokens. This design enables efficient parallel computation and improved modeling of long-range dependencies.

3 Formal Definition of the Implemented Algorithm

The implemented model is based on the original Transformer architecture introduced in the paper [1], with several modifications motivated by improved training stability and performance. In particular, the architecture adopts a pre-normalization scheme instead of the original post-normalization design and compares the use of absolute sinusoidal positional encodings with rotary positional embeddings (RoPE). These changes preserve the core encoder-decoder structure while theoretically improving convergence behavior and generalization.

The model follows a standard sequence-to-sequence Transformer design composed of an encoder and a decoder, each constructed from repeated attention and feed-forward blocks. Both components employ residual connections and the aforementioned layer normalization to stabilize training and improve gradient flow.

A diagram illustrating the architecture is shown in Figure 3.1. The individual components of the model are described in detail in the following subsections.

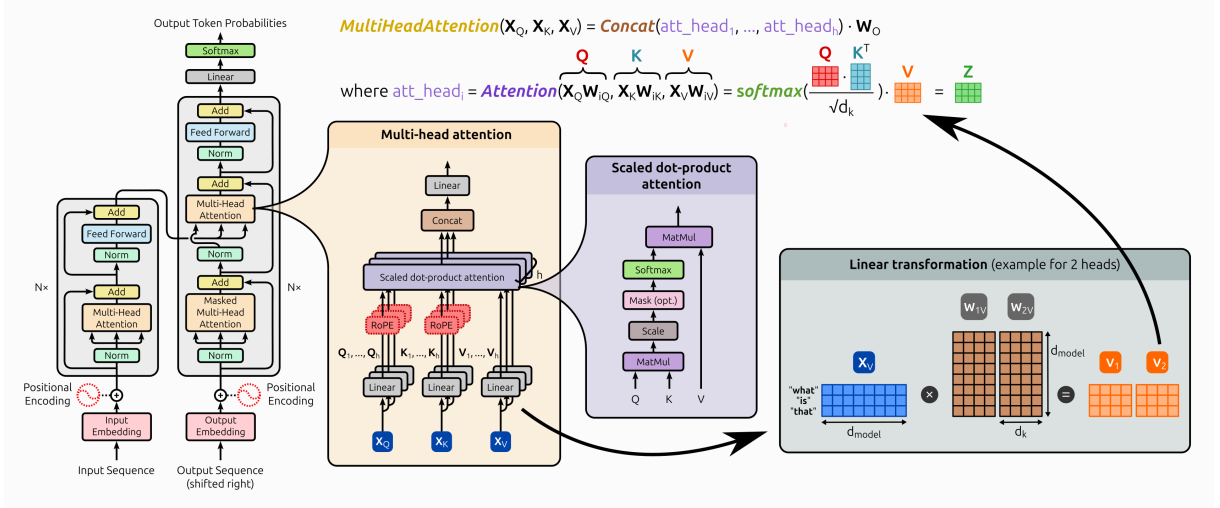


Figure 3.1: Overview of the Transformer encoder-decoder architecture used in this work (inspired by the original Transformer model from [1]).

3.1 Token Embedding and Positional Encoding

Each input token x_p , where p denotes its position in the sequence, is mapped to a continuous vector using an embedding matrix as

$$\mathbf{e}_p = \text{Embedding}(x_p) \in \mathbb{R}^{d_{\text{model}}}. \quad (3.1)$$

Since the Transformer has no inherent notion of order, positional information must be explicitly incorporated into the model. This is achieved using positional encoding techniques, which inject information about token positions into the model representations.

3.1.1 Sinusoidal Positional Encoding

The sinusoidal positional encoding proposed in [1] defines a deterministic transformation from token positions to continuous vectors

$$\text{PE}_{(p,2k)} = \sin\left(\frac{p}{10000^{\frac{2k}{d_{\text{model}}}}}\right), \quad \text{PE}_{(p,2k+1)} = \cos\left(\frac{p}{10000^{\frac{2k}{d_{\text{model}}}}}\right), \quad (3.2)$$

where p is the token position and k indexes the embedding dimension. This absolute positional encoding is added element-wise directly to the input token embeddings.

3.1.2 Rotary Positional Embedding (RoPE)

Rotary Positional Embedding [2] integrates both absolute and relative positional information by encoding positions directly into the self-attention mechanism. This is achieved by rotating the query and key vectors in two-dimensional subspaces, allowing attention scores to reflect relative token positions. Figure 3.2 demonstrates the application of the RoPE mechanism.

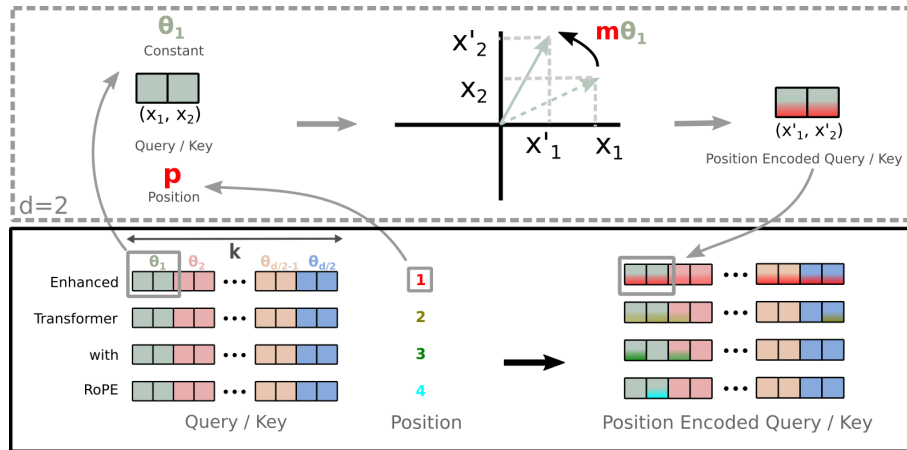


Figure 3.2: Illustration of Rotary Position Embedding (RoPE) [2].

Each pair of embedding dimensions is interpreted as a 2D vector and rotated by an angle that depends on the token position:

$$\theta_{p,k} = p \cdot 10000^{\frac{-2k}{d_{\text{model}}}}, \quad (3.3)$$

where p is the token position and k is the index of the feature pair in the representation. Each token representation consists of $\frac{d_{\text{model}}}{2}$ such pairs. For each feature pair, the rotation matrix is defined as

$$R(\theta_{p,k}) = \begin{bmatrix} \cos \theta_{p,k} & -\sin \theta_{p,k} \\ \sin \theta_{p,k} & \cos \theta_{p,k} \end{bmatrix}, \quad (3.4)$$

and finally, the rotated representation for each pair is obtained as

$$\begin{bmatrix} \tilde{x}_{p,2k} \\ \tilde{x}_{p,2k+1} \end{bmatrix} = R(\theta_{p,k}) \begin{bmatrix} x_{p,2k} \\ x_{p,2k+1} \end{bmatrix}. \quad (3.5)$$

Unlike absolute positional encodings, RoPE encodes relative positional information within the attention computation. The rotation is applied exclusively to the query and key vectors, leaving the value vectors unchanged. This design improves generalization to sequence lengths longer than those observed during training.

3.2 Scaled Dot-Product Attention

Given matrices for queries Q , keys K , and values V , scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V. \quad (3.6)$$

To incorporate attention masks, which are used to ignore padding tokens or prevent attending to future positions during decoding, the formula becomes

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V, \quad (3.7)$$

where matrix M contains 0 for allowed positions (tokens that should be attended to) and $-\infty$ (or a very large negative number) for masked positions (ignored ones). When these large negative values are passed through the softmax function, they become 0, effectively removing those tokens from the final weighted sum.

3.3 Multi-Head Attention

Multi-head attention projects the input into multiple subspaces. The Attention function refers to the scaled dot-product attention defined in the previous section. For i -th head:

$$\text{head}_i = \text{Attention}(X_Q W_i^Q, X_K W_i^K, X_V W_i^V), \quad (3.8)$$

where W_i^Q , W_i^K , and W_i^V are learnable projection matrices for queries, keys, and values. X_Q , X_K , and X_V are input representations. The outputs of all heads are concatenated and linearly transformed by the output matrix W^O as

$$\text{MultiHeadAttention}(X_Q, X_K, X_V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O. \quad (3.9)$$

3.4 Position-wise Feed-Forward Network

Each token representation is processed independently by a two-layer feed-forward network:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2. \quad (3.10)$$

3.5 Encoder

The encoder is composed of N identical layers, each consisting of a multi-head self-attention block followed by a position-wise feed-forward network. Residual connections and layer normalization are applied after each sublayer to stabilize training. Self-attention allows each token to attend to all other tokens in the input sequence, enabling the model to capture global contextual dependencies.

3.6 Decoder

The decoder is also composed of N identical layers, each containing three subcomponents: masked multi-head self-attention, multi-head Cross-attention, and a position-wise feed-forward network. Masked self-attention ensures that predictions at each position depend only on previously generated tokens, preserving the autoregressive property. Multi-head cross-attention allows the decoder to attend to the encoder outputs, with the decoder representations serving as queries and the encoder outputs serving as keys and values.

The final decoder representations are projected to vocabulary logits using a linear transformation, followed by a softmax operation to produce a probability distribution over the target vocabulary.

3.7 BLEU Evaluation Metric

Translation quality was evaluated using the BLEU (Bilingual Evaluation Understudy) metric [6], a widely used automatic evaluation measure for machine translation. BLEU estimates the similarity between a candidate translation sentence and one or more reference translations by computing the precision of matching n -grams, combined with a brevity penalty (BP) that penalizes overly short outputs.

First, the geometric mean of the modified n -gram precisions p_n is computed using n -grams up to order N , with positive weights w_n that sum to one. In practice, uniform weights $w_n = \frac{1}{N}$ are commonly used. For a given N , the BLEU- N score is defined as

$$\text{BLEU-}N = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right), \quad (3.11)$$

where p_n denotes the modified precision of n -grams. Intuitively, this precision measures the fraction of n -grams in the candidate translation that also appear in the reference translation. The brevity penalty BP is defined as

$$\text{BP} = \begin{cases} 1, & c > r, \\ \exp \left(1 - \frac{r}{c} \right), & c \leq r, \end{cases} \quad (3.12)$$

where c and r denote the total lengths of the candidate and reference translations, respectively.

In this work, BLEU-1, BLEU-2, and BLEU-3 scores are used to capture both lexical accuracy and short-range fluency. All scores are computed using greedy decoding to ensure consistent evaluation across experiments.

4 Implementation and Training Setup

4.1 Implementation Details and Software Tools

An intuitive understanding of the Transformer architecture was further supported by the visual explanations in [7]. The model was implemented in Python using the PyTorch framework. All components of the encoder–decoder architecture, including multi-head attention, positional encodings, masking, and autoregressive inference, were implemented using only basic neural network primitives such as linear layers and layer normalization. No high-level Transformer abstractions were used.

A custom word-level tokenizer was implemented from scratch. Subword tokenization using Byte Pair Encoding (BPE) was supported through the HuggingFace Tokenizers library [4].

Training was performed using teacher forcing, while inference used greedy autoregressive decoding. Model evaluation was handled by a separate testing script that computes BLEU- N scores to assess translation quality.

The main contribution of this work is the full from-scratch implementation of the Transformer encoder–decoder architecture, along with the integration and comparison of sinusoidal positional encoding with rotary positional embedding, and the demonstration of its sequence-to-sequence functionality on standard tasks.

4.2 Training

Two Transformer models were trained: a larger model ($T\text{-Main}$) and a compact model ($T\text{-Tiny}$). $T\text{-Main}$ was trained for 40 epochs, while $T\text{-Tiny}$ was trained for 200 epochs with a greater batch size. Training utilized computational resources provided by Metacentrum¹. The model details, hyperparameters, and training setup are summarized in Table 1.

Parameter	Value ($T\text{-Main}$ / $T\text{-Tiny}$)
Model parameters	18.9M / 1.5M
Embedding dimension (d_{model})	256 / 32
Number of heads	4 / 2
Encoder layers	4 / 2
Decoder layers	4 / 2
Feed-forward dimension	1024 / 256
Optimizer	Adam
Learning rate	7.5×10^{-4} / 1.0×10^{-3}
Batch size	512 / 1024
Loss function	Cross-Entropy

Table 1: Architecture, hyperparameters and configuration of $T\text{-Main}$ and $T\text{-Tiny}$ models.

¹MetaVO Metacentrum: <https://metavo.metacentrum.cz/>

5 Testing and Evaluation

5.1 Machine Translation (Czech to English) task

The primary evaluation task for the implemented Transformer model is machine translation from Czech to English. The model was trained in a sequence-to-sequence setting, where the encoder processes a source sentence in Czech and the decoder generates the corresponding English translation. During training, efficient teacher forcing was applied to provide the decoder with the ground-truth previous tokens. However, during inference, the decoder generates translations autoregressively using greedy decoding, relying only on previously predicted tokens at each step.

Translation quality was evaluated on multiple test sets from Czech–English parallel corpora from the OPUS collection [3]. Table 2 summarizes the datasets, including the number of training and test samples, and example sentences from each dataset show their typical content and style.

Dataset	Description	Train	Test
TED2020	TED Talks	140k	7k
WMT19	Online News	–	30k
Tatoeba	Simple phrases	–	60k

Table 2: Datasets used for training and evaluation for the translation task.

TED2020 (spoken, long-form sentences)

- **CS:** Představte si žonglování se sněhovou koulí v tropech.
EN: Imagine juggling a snowball across the tropics.
- **CS:** Myslím tím to, že pravděpodobnost a statistika jsou matematikou her a sázení.
EN: I mean, probability and statistics, it’s the mathematics of games and gambling.

WMT19 (formal news-style text)

- **CS:** Ekonomika v letošním třetím čtvrtletí meziročně vzrostla o 1,5 procenta.
EN: In this year’s third quarter, the economy showed an interim increase of 1.5%.
- **CS:** Nad Norskem bylo možné pozorovat zvláštní světelný efekt neznámého původu.
EN: Strange lights of unknown origin have been seen over Norway.

Tatoeba (short, simple sentences)

- **CS:** Dejme tomu, že bys mohl cestovat, kam bys jel nejdříve?
EN: Supposing you could travel, where would you go first?
- **CS:** Každý student má volný přístup do knihovny.
EN: Every student has free access to the library.

5.1.1 Results

Translation quality was evaluated using BLEU-1 to BLEU-3 scores computed on the test datasets using greedy decoding. The quantitative results for both models and positional encoding strategies are presented in Table 3. It should be noted that the WMT19 and Tatoeba datasets contain completely distinct types of sentences from those used for training on the TED2020 dataset, reflecting different linguistic styles and complexity.

Across all datasets, the use of rotary positional embedding consistently yields slightly higher BLEU scores compared to standard sinusoidal positional encoding. While the improvements are relatively modest, this behavior is expected given that experiments were conducted with a maximum sequence length of 150 tokens. Under such constraints, the benefits of rotary positional encoding are limited, as long-range dependencies occur less frequently. It is therefore reasonable to expect that the performance gap between RoPE and sinusoidal encodings would become more significant when training on longer sequences or larger, more diverse datasets.

A clear difference in performance can also be seen between the two model sizes. The robust *T-Main* model consistently outperforms the lightweight *T-Tiny* across all datasets and metrics. This confirms that increased model capacity (through higher embedding dimensionality and deeper encoder-decoder stacks) improves the model’s ability to capture syntactic and semantic relationships. Despite its smaller size, *T-Tiny* still achieves reasonable performance, indicating that the core Transformer architecture remains effective even under tight parameter constraints.

Overall, these results illustrate that both positional encoding and model capacity influence translation quality. RoPE consistently provides a small but meaningful improvement over sinusoidal encoding, and larger models benefit from better generalization and higher BLEU scores under the evaluated experimental conditions.

Model	Encoding	WMT19		Tatoeba		TED2020	
		BLEU-1	BLEU-3	BLEU-1	BLEU-3	BLEU-1	BLEU-3
<i>T-Main</i>	RoPE	0.38	0.11	0.57	0.27	0.60	0.37
	Sinusoidal	0.38	0.10	0.56	0.26	0.59	0.35
<i>T-Tiny</i>	RoPE	0.31	0.06	0.46	0.16	0.44	0.17
	Sinusoidal	0.29	0.05	0.44	0.14	0.42	0.15

Table 3: Translation results across datasets and model variants.

5.1.2 Example predictions

In addition to quantitative scores, qualitative inspection of model outputs provides insight into translation correctness. The following examples illustrate typical translations produced by the model on test data, together with their reference translations and corresponding BLEU scores.

It can be observed that some examples receive relatively low scores (e.g. Example III) despite being semantically correct and fluent translations. This highlights a known limitation of BLEU-based evaluation when only a single reference translation is available, as alternative valid phrasings may be penalized even though the correct meaning is preserved.

Example I

Source: Taková je bohužel realita, ve které musím žít.

Prediction: This is unfortunately the reality I have to live in.

Reference: That's the reality, unfortunately, in which I have to live.

BLEU-1: 0.682, **BLEU-2:** 0.477, **BLEU-3:** 0.348

Example II

Source: Nechtěla jsem říct, že to bude můj poslední závod.

Prediction: I didn't want to tell you that this is my last race.

Reference: I didn't want to say this was my last race.

BLEU-1: 0.692, **BLEU-2:** 0.588, **BLEU-3:** 0.501

Example III

Source: Co se tam děje?

Prediction: What's going on there?

Reference: What is happening there?

BLEU-1: 0.400, **BLEU-2:** 0.316, **BLEU-3:** 0.000

Example IV

Source: Tom je jediný, kdo má fotoaparát.

Prediction: Tom is the only person who has a camera.

Reference: Tom is the only one with a camera.

BLEU-1: 0.700, **BLEU-2:** 0.624, **BLEU-3:** 0.526

5.2 Question Answering (QA) task

To evaluate the generality of the implemented Transformer beyond machine translation, an additional experiment was conducted on the bAbI Question Answering dataset [5]. The bAbI dataset consists of 20 synthetic QA tasks designed to test simple text understanding and basic reasoning abilities such as deduction and counting. In total, it contains approximately 200,000 training samples and 20,000 test samples, as shown in Table 4.

Each sample consists of a context paragraph, a question, and an answer. For this task, the encoder input is formed by concatenating the context and the question into a single sequence.

Dataset	Description	Train	Test
bAbI	Logical reasoning tasks	200k	20k

Table 4: Datasets used for training and evaluation for the QA task.

The model was trained for 50 epochs with the same settings as the *T-Main* model. No task-specific metrics were computed. Instead, a qualitative evaluation was performed. The model successfully predicted many test samples, as you can see in the examples below.

5.2.1 Example predictions

Example I

Context: The bedroom is west of the bathroom. The bathroom is west of the office.

Question: What is the bathroom east of?

Prediction: bedroom

Reference: bedroom

Example II

Context: Sandra is no longer in the kitchen. John is in the hallway. Daniel travelled to the kitchen. Mary is in the office.

Question: Is John in the kitchen?

Prediction: no

Reference: no

Example III

Context: Mary got the apple there. John went back to the kitchen. John moved to the garden. Mary left the apple there.

Question: How many objects is Mary carrying?

Prediction: none

Reference: none

Example IV

Context: Lily is a frog. Brian is a lion. Greg is a rhino. Julius is a rhino. Julius is white. Brian is green. Lily is green. Greg is white. Bernhard is a frog.

Question: What color is Bernhard?

Prediction: green

Reference: green

These examples demonstrate that the model is capable of handling diverse reasoning patterns present in the bAbI dataset, including yes/no questions, object tracking, and attribute inference. These results indicate that the implemented Transformer is able to generalize beyond machine translation to other sequence-to-sequence tasks.

6 Discussion and Conclusions

This project reveals that a complete Transformer encoder–decoder model can be implemented from scratch using basic neural network building blocks. The resulting implementation successfully learns meaningful attention patterns and achieves reasonable performance on sequence-to-sequence tasks, such as machine translation and question answering, even at relatively small model scales.

A comparison of positional encoding strategies shows that Rotary Positional Embedding consistently outperforms absolute sinusoidal positional encoding across the evaluated tasks and datasets, although the observed differences are subtle due to sequence length and complexity constraints. This suggests that the advantages of RoPE would become more pronounced for longer sequences and more complex dependencies.

Overall, the project fulfills its educational objectives by providing a thorough understanding of the Transformer architecture, formal modeling of sequence-to-sequence learning, and practical experience in implementing and evaluating modern attention-based models.

References

- [1] Vaswani, A., et al. *Attention Is All You Need*. NeurIPS, 2017. <https://arxiv.org/abs/1706.03762>
- [2] Su, J., et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021. <https://arxiv.org/abs/2104.09864>
- [3] OPUS – Open Parallel Corpora. <https://opus.nlpl.eu>
- [4] HuggingFace. *Tokenizers: Fast State-of-the-Art Tokenizers Optimized for Research and Production*. <https://huggingface.co/docs/tokenizers/index>
- [5] Weston, J., Bordes, A., Chopra, S., Mikolov, T. *bAbI: Towards AI-Complete Question Answering*. arXiv:1502.05698, 2015. https://huggingface.co/datasets/facebook/babi_qa
- [6] Papineni, K., Roukos, S., Ward, T., Zhu, W.-J. *BLEU: A Method for Automatic Evaluation of Machine Translation*. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002.
- [7] J. Ferrer, *How Transformers Work: A Detailed Exploration of Transformer Architecture*, DataCamp, 2024. Available at: <https://www.datacamp.com/tutorial/how-transformers-work>.